

Multi Pipeline ETL Framework for Web Server Log Analytics

Status Report + Architecture

Team Members

Ayush Mishra (IMT2023129)

Harsh Sinha (IMT2023571)

Syed Naveed Mohammed (IMT2023119)

Santhosh Vodnala (IMT2023622)

GitHub Repository

Multi Pipeline ETL Framework Project

Project Demonstration Video

YouTube Project Demonstration Link

May 18, 2026

Contents

1	Introduction	3
2	Proposed Architecture and Workflow	3
2.1	System Overview	3
2.2	End-to-End Data Flow	5
2.3	Pipeline Backends	6
2.4	Design Principles	7
3	Parsing Strategy and Batching Approach	7
3.1	Log Format and Structure	7
3.2	Parsing Strategy	7
3.2.1	Regular Expression	8
3.2.2	Field Extraction and Transformation	8
3.3	Batching Approach	9
3.3.1	MongoDB:	9
3.3.2	BatchReader — Streaming Lines from Disk	9
3.3.3	BatchProcessor — Parsing a Batch	10
3.3.4	MapReduce:	10
3.3.5	Batch Processing using Hadoop MapReduce	10
3.3.6	Mapper Initialization	11
3.3.7	Record Parsing and Validation	11
3.3.8	Writing Structured Batch Outputs	12
3.3.9	Batching and Parallel Execution	13
3.3.10	Pig:	13
3.3.11	Hive:	15
3.3.12	Batch Processing using Apache Hive	15
3.3.13	Raw Data Staging	16
3.3.14	Batch Storage in Raw Log Table	16
3.3.15	Regex-Based Parsing and Validation	17
3.3.16	Malformed Record Separation	18
3.3.17	Generation of Structured Filtered Logs	18
3.3.18	Batching Strategy in Hive	19
3.4	Handling Malformed Records	20
3.5	Pipeline Summary Output	20

4	Relational Reporting Database Schema	21
4.1	Schema Design	21
4.1.1	Run Metadata Table	21
4.1.2	Batch Metadata Table	21
4.1.3	Query Result Tables	22
4.1.4	Malformed Record Summary Table	23
5	Query Outputs and Demonstrations	23
5.1	Storage in PostgreSQL	23
5.2	Query 1: Daily Traffic Summary	24
5.2.1	Implementation Details	24
5.2.2	Sample Output	25
5.2.3	Observations	25
5.3	Query 2: Top Requested Resources	26
5.3.1	Implementation Details	26
5.3.2	Sample Output	27
5.3.3	Observations	27
5.4	Query 3: Hourly Error Analysis	27
5.4.1	Implementation Details	28
5.4.2	Sample Output	29
5.4.3	Observations	29
6	Runtime Module	30
7	Pipeline Performance Comparison	31
8	Conclusion	32

1. Introduction

This report presents the implementation status and architecture of a multi-pipeline ETL framework for web server log analysis. It highlights the current progress, system design, and workflow used to process and aggregate log data using one of the four pipelines of the user's choice - MapReduce, MongoDB, Hive, and Pig, along with the storage of results in PostgreSQL.

2. Proposed Architecture and Workflow

2.1 System Overview

The system implements a unified multi-pipeline ETL and reporting framework for processing large-scale web server logs. It supports four interchangeable execution backends: MapReduce, MongoDB, Apache Hive, and Apache Pig.

Raw log data is transformed into a structured representation, processed in batches, and aggregated using the selected execution backend. The generated outputs, along with runtime and batch metadata, are finally stored in PostgreSQL for centralized reporting and comparative analysis.

The modular architecture improves scalability, maintainability, and extensibility, allowing additional execution backends or analytical queries to be integrated with minimal changes to the overall system.

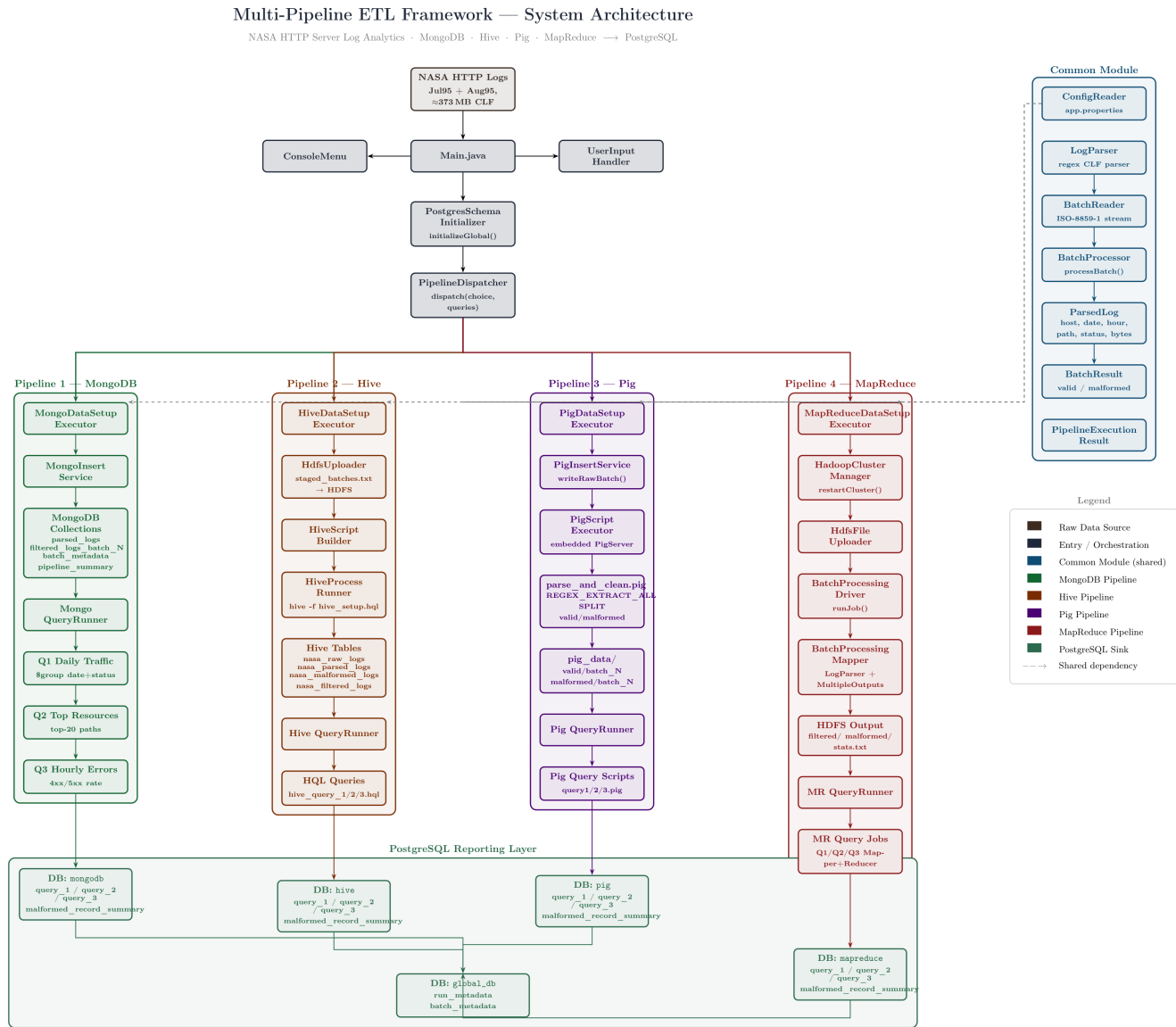


Figure 1: Architecture

2.2 End-to-End Data Flow

```
=====
      NOSQL ETL PIPELINE FRAMEWORK
=====

AVAILABLE PIPELINES:
1. MongoDB
2. Hive
3. Pig
4. MapReduce
5. Run All Pipelines

Select pipeline number: 3

AVAILABLE QUERIES:
1. Daily Traffic Analysis
2. Top Requested Resources
3. Hourly Error Analysis

Enter queries to execute (space separated): 1

Current batch.size = 25000

Enter batch size (press Enter to use default = 25000): 120000

Updated app.properties:
batch.size=120000
Database already exists: global_db

=====
Running Pig Pipeline...
=====
```

Figure 2: The CLI interface wherein the user can choose the pipeline to run, which queries to run as well as batch size

The user is provided with a CLI interface through which one of the four supported execution pipelines can be selected. The selected pipeline then executes the complete ETL workflow and stores the analytical results and execution metadata in PostgreSQL tables.

The data flows through the following stages:

- **Input Layer:** Raw NASA log files are read from disk.
- **Parsing Layer:** Logs are converted into structured records containing fields such as date, status code, and bytes.
- **Batching Layer:** Records are grouped into batches to enable scalable and parallel processing.
- **Processing Layer:** Aggregation queries are executed on each batch collection using the respective pipelines.
- **Global Aggregation:** Partial results from all batches are merged to produce final aggregated results.
- **Output Layer:** Final results are enriched with metadata and stored in PostgreSQL for reporting.

2.3 Pipeline Backends

The system supports four interchangeable execution backends: MapReduce, MongoDB, Apache Hive, and Apache Pig. The user can select any one of these pipelines at runtime through the command-line interface.

Each pipeline executes the same ETL workflow and analytical queries while preserving identical semantics across all implementations. This ensures fair comparison between different data processing paradigms in terms of implementation style, runtime behaviour, batching efficiency, and reporting suitability.

- **MapReduce Pipeline:** Executes parsing, transformation, and aggregation using distributed MapReduce jobs.
- **MongoDB Pipeline:** Uses MongoDB aggregation pipelines to process semi-structured log data efficiently.
- **Hive Pipeline:** Performs ETL and analytical processing using HiveQL queries over structured log tables stored in Hadoop.
- **Pig Pipeline:** Implements the workflow using Apache Pig scripts with dataflow-based transformations and aggregations.

Batch-level processing is executed independently within the selected backend. After batch execution, the intermediate aggregation outputs are merged to generate the final global results. The final outputs, along with execution metadata such as runtime, batch information, and pipeline details, are then loaded into PostgreSQL for reporting and comparison.

2.4 Design Principles

The system is designed based on the following principles:

- **Modularity:** Parsing, batching, and query execution are implemented as independent components.
- **Scalability:** Batch-based processing and parallel execution enable handling of large datasets.
- **Extensibility:** The architecture can be extended to support additional execution backends and analytical workloads in future phases.
- **Consistency:** Aggregation logic ensures accurate results across all batches.

3. Parsing Strategy and Batching Approach

This section describes how raw NASA HTTP server log files are read, parsed into structured records, grouped into batches for processing, and how malformed entries are handled throughout the pipeline. The implementation is spread across four classes in the `common` module: `LogParser`, `BatchProcessor`, `BatchReader`, and `BatchResult`.

3.1 Log Format and Structure

The input dataset consists of two raw NASA HTTP server access log files: `NASA_access_log_Jul95` and `NASA_access_log_Aug95`, together totalling approximately 373 MB of uncompressed log data.

Each line in the log follows the Combined Log Format (CLF), a standard format used by Apache and NCSA HTTP servers. A representative log line is shown below:

Listing 1: Sample NASA HTTP Log Entry

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo
/ HTTP/1.0" 200 6245
```

The fields present in each log entry are as follows:

3.2 Parsing Strategy

Parsing is handled entirely by the `LogParser` class (`common/src/main/java/com/example/util/LogParser.java`).

The parser applies a single compiled regular expression to each raw log line and extracts all relevant fields in one pass.

Table 1: NASA HTTP Log Fields

Field	Example Value	Description
Host	199.72.81.55	Client hostname or IP address
Ident	-	RFC 1413 identity (always -)
Auth User	-	Authenticated username (always -)
Timestamp	01/Jul/1995:00:00:01 -0400	Request date and time with timezone
Request	GET /history/apollo/ HTTP/1.0	Full HTTP request line
Status Code	200	HTTP response status code
Bytes	6245	Response size in bytes (- if unknown)

3.2.1 Regular Expression

The core pattern used is:

Listing 2: Log Parsing Regex Pattern

```
1 ^(\S+) \S+ \S+ \[(.*?)\] "(.*)" (\d{3}) (\S+)$
```

The five capture groups in this pattern are described in the table below:

Table 2: Regex Capture Groups and Their Meanings

Group	Pattern	Field	Description
1	(\S+)	Host	Non-whitespace token — client IP or hostname
2	(.*?)	Raw Timestamp	Content inside [] — full date-time string
3	(.*?)	Request	Content inside " " — HTTP method, path, protocol
4	(\d{3})	Status Code	Exactly 3 digits — HTTP response code
5	(\S+)	Bytes	Non-whitespace — response size or -

3.2.2 Field Extraction and Transformation

Once the regex matches, the following transformations are applied before populating a `ParsedLog` object:

- **Timestamp parsing:** The raw timestamp (e.g. 01/Jul/1995:00:00:01 -0400) is split on ":" to separate the date part from the hour. The date is parsed using `DateTimeFormatter` with the pattern `dd/MMM/yyyy` and then re-formatted to ISO-8601 (`yyyy-MM-dd`) for downstream storage. The hour is extracted as an integer directly.
- **Request line parsing:** The request string (e.g. GET /history/apollo/ HTTP/1.0) is split on whitespace into method, path, and protocol. If fewer than three tokens are found, the record is flagged as malformed.
- **Bytes field:** If the bytes field equals "-", it is stored as 0. Otherwise it is parsed as a `long`.

- **Batch ID injection:** Each `ParsedLog` record is stamped with the `batchId` of the batch it belongs to, enabling per-batch filtering and aggregation downstream.

The resulting `ParsedLog` model captures the following fields:

Listing 3: `ParsedLog` Model Fields

```
1 private String host;  
2 private String rawTimestamp;  
3 private String date;           // ISO-8601 format: yyyy-MM-dd  
4 private int    hour;           // 0-23  
5 private String method;         // GET, POST, HEAD, etc.  
6 private String path;           // Requested resource path  
7 private String protocol;       // HTTP/1.0 or HTTP/1.1  
8 private int    status;         // HTTP status code  
9 private long   bytes;          // Response size (0 if unknown)  
10 private boolean malformed;     // true if parsing failed  
11 private int    batchId;        // Batch this record belongs to
```

3.3 Batching Approach

Large log files are not loaded entirely into memory. Instead, they are streamed and consumed in fixed-size batches. This design keeps memory usage bounded and naturally parallelizes with the query execution layer.

3.3.1 MongoDB:

3.3.2 BatchReader — Streaming Lines from Disk

The `BatchReader` class wraps a `BufferedReader` opened with ISO-8859-1 encoding (required because the NASA logs contain non-UTF-8 byte sequences). It exposes a single method, `readNextBatch(int batchSize)`, which reads up to `batchSize` lines from the file and returns them as a `List<String>`. An empty list signals end-of-file. `BatchReader` implements `AutoCloseable` so it is safely used inside a `try-with-resources` block.

Listing 4: `BatchReader` Core Method

```
1 public List<String> readNextBatch(int batchSize) throws  
   IOException {  
2     List<String> lines = new ArrayList<>(batchSize);  
3     String line;  
4     while (lines.size() < batchSize &&  
5         (line = reader.readLine()) != null) {  
6         lines.add(line);  
7     }  
}
```

```
8     return lines;
9 }
```

3.3.3 BatchProcessor — Parsing a Batch

The `BatchProcessor` class receives a list of raw lines and a `batchId`, calls `LogParser.parse()` on every line, counts valid and malformed records, and returns a `BatchResult` containing all `ParsedLog` objects plus the summary counters.

Listing 5: BatchProcessor Core Method

```
1 public static BatchResult processBatch(
2     List<String> rawLines, int batchId) {
3
4     List<ParsedLog> parsedLogs = new ArrayList<>(rawLines.size());
5     int malformedCount = 0;
6
7     for (String line : rawLines) {
8         ParsedLog log = LogParser.parse(line, batchId);
9         if (log.isMalformed()) malformedCount++;
10        parsedLogs.add(log);
11    }
12
13    return BatchResult.builder()
14        .batchId(batchId)
15        .totalRecords(rawLines.size())
16        .malformedRecords(malformedCount)
17        .validRecords(rawLines.size() - malformedCount)
18        .parsedLogs(parsedLogs)
19        .build();
20 }
```

3.3.4 MapReduce:

3.3.5 Batch Processing using Hadoop MapReduce

The MapReduce pipeline performs parsing, validation, and batch-level processing using a Hadoop Mapper-based execution model. Raw NASA HTTP log records are streamed from HDFS and processed independently by multiple mapper tasks in parallel.

The core batch processing logic is implemented in the `BatchProcessingMapper` class. Each mapper receives raw log lines as input, parses them using the shared `LogParser` utility, classifies records as valid or malformed, and writes the processed outputs into separate HDFS directories using `Hadoop MultipleOutputs`.

This design enables scalable distributed processing while preserving the same parsing semantics used across the other execution pipelines.

3.3.6 Mapper Initialization

During mapper initialization, a `MultipleOutputs` instance is created to support simultaneous writing of valid and malformed records into separate output paths. In addition, every mapper obtains a unique mapper identifier using the Hadoop task context.

Listing 6: Mapper Setup Method

```
1 @Override
2 protected void setup(Context context) {
3
4     multipleOutputs =
5         new MultipleOutputs<>(context);
6
7     mapperId =
8         context
9             .getTaskAttemptID()
10            .getTaskID()
11            .toString();
12 }
```

3.3.7 Record Parsing and Validation

For each input log line, the mapper performs the following operations:

- Removes leading and trailing whitespace.
- Updates Hadoop counters for total processed records.
- Parses the log entry using the common `LogParser` utility.
- Classifies the record as valid or malformed.
- Writes malformed records into a dedicated malformed output directory.
- Writes valid structured records into the filtered output directory.

The parsing and classification logic is implemented as follows:

Listing 7: Core Mapper Processing Logic

```
1 ParsedLog log =
2     LogParser.parse(
3         line,
```

```
4         0
5     );
6
7     if (log.isMalformed()) {
8
9         context.getCounter(
10             LogCounters.TOTAL_MALFORMED
11         ).increment(1);
12
13         multipleOutputs.write(
14             NullWritable.get(),
15             new Text(
16                 mapperId + "\t" + line
17             ),
18             malformedPath
19         );
20
21         return;
22     }
23
24     context.getCounter(
25         LogCounters.TOTAL_VALID
26     ).increment(1);
```

3.3.8 Writing Structured Batch Outputs

After successful parsing, the extracted fields are converted into a structured tab-separated format and written into the filtered HDFS output directory.

Listing 8: Structured Output Generation

```
1 String parsedLine =
2
3     log.getHost() + "\t" +
4         log.getRawTimestamp() + "\t" +
5         log.getDate() + "\t" +
6         log.getHour() + "\t" +
7         log.getMethod() + "\t" +
8         log.getPath() + "\t" +
9         log.getProtocol() + "\t" +
10        log.getStatus() + "\t" +
11        log.getBytes();
12
```

```
13 multipleOutputs.write(  
14     NullWritable.get(),  
15     new Text(parsedLine),  
16     filteredPath  
17 );
```

3.3.9 Batching and Parallel Execution

In the MapReduce pipeline, batching is naturally handled through Hadoop input splits and parallel mapper execution. Each mapper processes a subset of the input dataset independently, enabling distributed execution across multiple processing tasks.

The use of Hadoop counters allows the system to track:

- Total processed records
- Total valid records
- Total malformed records

The filtered outputs and malformed outputs generated by all mapper tasks are later aggregated and used as input for the analytical query jobs.

This batching strategy improves scalability, fault isolation, and processing throughput while preserving consistency with the ETL semantics implemented across all other pipelines.

3.3.10 Pig:

The Pig Pipeline, similar to MongoDB, reads and batches the log files using BatchReader object. This object gives us the next batch as a list of strings. It is basically a list of unparsed logs. The BatchReader makes sure that the list is of size *batch.size* (If there are fewer logs available, it returns the entirety of the remaining logs).

Once, a batch is returned by BatchReader, we store it as *".pig_data/raw/batch_{batchId}.txt"*. This txt file is then processed by *parse_and_clean.pig* to give us validated logs and malformed logs of that batch. The pig latin script first loads the batch logs, and tries to fit them into the regex specified. If the log matches the regex, it is a valid log, else it is malformed. We do further processing with the valid log before storing it appropriately. The implementation details of the pig latin script is given below:

Listing 9: parse_and_clean.pig code

```
1 -- loading the raw batch
2 raw_data = LOAD '$INPUT_FILE' USING TextLoader() AS (line:
   chararray);
3
4 -- Parse the line while preserving original input
5 parsed = FOREACH raw_data GENERATE
6     line,
7     REGEX_EXTRACT_ALL(
8         line,
9         '^(\S+) \S+ \S+ \[([.*?)]' "(.*)" (\d{3}) (\S+)$'
10    ) AS fields;
11
12 -- Split valid and malformed records
13 SPLIT parsed INTO
14     valid_logs IF fields IS NOT NULL,
15     malformed_logs IF fields IS NULL;
16
17 -- Extract parsed fields from valid logs
18 extracted_valid = FOREACH valid_logs GENERATE
19     line,
20     (chararray)fields.$0 AS host,
21     (chararray)fields.$1 AS rawTimestamp,
22     (chararray)fields.$2 AS request,
23     (int)fields.$3 AS status,
24     (chararray)fields.$4 AS bytes;
25
26 -- Split timestamp and request parts
27 split_valid = FOREACH extracted_valid GENERATE
28     host,
29     rawTimestamp,
30     STRSPLIT(rawTimestamp, ':') AS date_parts,
31     STRSPLIT(TRIM(REPLACE(request, '\\s+', ' ')), ' ') AS
        req_parts,
32     status,
33     bytes,
34     (int) '$BATCH_ID' AS batchId;
35
36 -- Format final cleaned output
37 formatted_valid = FOREACH split_valid GENERATE
38     host,
```

```

39     rawTimestamp ,
40     ToString(
41         ToDate((chararray)date_parts.$0 , 'dd/MMM/yyyy'),
42         'yyyy-MM-dd'
43     ) AS formattedDate ,
44
45     (int)(date_parts.$1) AS hour ,
46
47     (SIZE(req_parts) > 0 ? (chararray)req_parts.$0 : '') AS method
48     ,
49     (SIZE(req_parts) > 1 ? (chararray)req_parts.$1 : '') AS path ,
50
51     (SIZE(req_parts) > 2 ? (chararray)req_parts.$2 : '') AS
52     protocol ,
53
54     status ,
55
56     (bytes == '-' ? 0L : (long)bytes) AS bytes_val ,
57
58     batchId;
59
60 -- Keep malformed raw lines separately
61 just_malformed = FOREACH malformed_logs GENERATE line;
62
63 -- Store outputs
64 STORE formatted_valid INTO '$VALID_OUTPUT' USING PigStorage('\t');
65 STORE just_malformed INTO '$MALFORMED_OUTPUT' USING PigStorage();
66 );

```

3.3.11 Hive:

3.3.12 Batch Processing using Apache Hive

The Hive pipeline performs large-scale ETL processing using HiveQL-based transformations executed over data stored in HDFS. The pipeline follows a staged batch-processing architecture where raw NASA HTTP log records are first loaded into staging tables and then transformed into structured analytical tables.

The implementation is designed to preserve the same parsing semantics used across the MongoDB, Pig, and MapReduce pipelines while leveraging Hive's distributed SQL

execution model for scalable analytics.

3.3.13 Raw Data Staging

The pipeline begins by creating a temporary staging table used as a landing zone for batch ingestion from HDFS.

Listing 10: Hive Raw Staging Table

```
1 CREATE TABLE nasa_raw_stage (  
2     batch_id INT,  
3     raw_line STRING  
4 )  
5 ROW FORMAT DELIMITED  
6 FIELDS TERMINATED BY '\000'  
7 STORED AS TEXTFILE;
```

Pre-processed staged batches are then loaded into the staging table directly from HDFS.

Listing 11: Loading Staged Data from HDFS

```
1 LOAD DATA INPATH '${hiveconf:hdfsBase}/nasa_raw/staged'  
2 OVERWRITE INTO TABLE nasa_raw_stage;
```

This staging strategy allows the pipeline to process log data in batch-oriented chunks while maintaining compatibility with distributed HDFS storage.

3.3.14 Batch Storage in Raw Log Table

After staging, the records are transferred into the main raw log table stored in Parquet format for optimized analytical access.

Listing 12: Raw Log Table Creation

```
1 CREATE TABLE nasa_raw_logs (  
2     raw_line STRING,  
3     batch_id INT  
4 ) STORED AS PARQUET;
```

The staged records are inserted into the raw log table using an `INSERT OVERWRITE` operation.

Listing 13: Loading Raw Logs

```
1 INSERT OVERWRITE TABLE nasa_raw_logs  
2 SELECT raw_line, batch_id  
3 FROM nasa_raw_stage;
```

Using Parquet storage improves query performance and reduces storage overhead during downstream analytics execution.

3.3.15 Regex-Based Parsing and Validation

The pipeline then parses raw log records using Hive regular expression extraction functions. The parsing logic extracts:

- host
- timestamp
- request
- status code
- and byte count

The parsed values are stored in the `nasa_parsed_logs` table.

Listing 14: Regex-Based Parsing Logic

```

1 INSERT OVERWRITE TABLE nasa_parsed_logs
2 SELECT
3     batch_id,
4     raw_line,
5
6     regexp_extract(
7         RTRIM(raw_line),
8         '^((\\S+)\\S+\\S+\\[(.\\*?)\\]\"(.\\*?)\"(\\d{3})\\S+)$',
9         1
10    ) AS host,
11
12    regexp_extract(
13        RTRIM(raw_line),
14        '^((\\S+)\\S+\\S+\\[(.\\*?)\\]\"(.\\*?)\"(\\d{3})\\S+)$',
15        2
16    ) AS raw_timestamp
17
18 FROM nasa_raw_logs;
```

This parsing strategy ensures consistency with the shared Common Log Format parsing semantics implemented across all pipelines.

3.3.16 Malformed Record Separation

Malformed records are isolated into a dedicated Hive table for error tracking and reporting.

Listing 15: Malformed Record Extraction

```
1 INSERT OVERWRITE TABLE nasa_malformed_logs
2 SELECT
3     batch_id,
4     raw_line
5 FROM nasa_parsed_logs
6 WHERE
7     host = '' OR host IS NULL;
```

This allows malformed records to be analyzed independently without affecting downstream analytics queries.

3.3.17 Generation of Structured Filtered Logs

After validation, only valid records are transformed into a structured analytical format and stored in the `nasa_filtered_logs` table.

The transformation extracts:

- log date
- hour
- HTTP method
- resource path
- protocol
- status code
- and byte count

Listing 16: Filtered Structured Log Generation

```
1 INSERT OVERWRITE TABLE nasa_filtered_logs
2 SELECT
3     batch_id,
4     host,
5     raw_timestamp,
6
7     split(raw_timestamp, ':')[0] AS log_date_raw,
```

```
9      CAST(  
10          split(raw_timestamp, ':')[1] AS INT  
11      ) AS log_hour,  
12  
13      CASE  
14          WHEN size(split(TRIM(request), ' ')) >= 3  
15          THEN split(TRIM(request), ' ')[0]  
16          ELSE ''  
17      END AS method,  
18  
19      CAST(status_str AS INT) AS status  
20  
21 FROM nasa_parsed_logs  
22 WHERE  
23     host != '' AND host IS NOT NULL;
```

The resulting filtered table becomes the primary analytical dataset used for all Hive query execution.

3.3.18 Batching Strategy in Hive

The Hive pipeline implements batching through staged HDFS ingestion combined with table-based transformation workflows.

Each batch is:

- staged into HDFS,
- loaded into Hive staging tables,
- parsed using distributed HiveQL execution,
- validated using regex-based filtering,
- and transformed into structured Parquet tables.

The use of Hive allows distributed execution of large-scale ETL operations while maintaining consistency with the parsing, validation, and malformed-record handling semantics implemented across the MongoDB, Pig, and MapReduce pipelines.

This batching architecture improves:

- scalability,
- storage efficiency,
- fault isolation,
- and query execution performance.

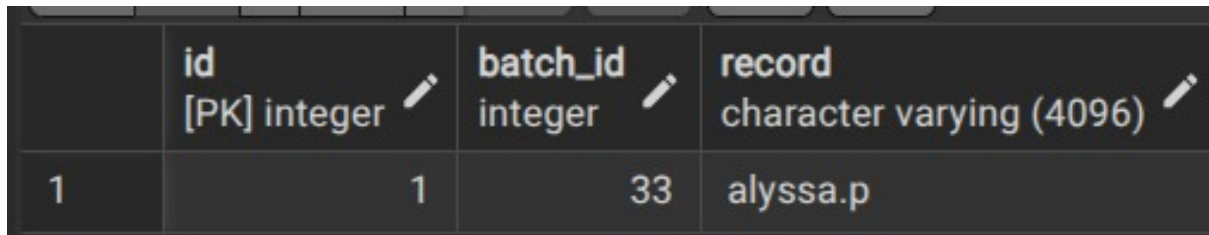
3.4 Handling Malformed Records

A record is classified as malformed under any of the following conditions:

- The raw log line does not match the regex pattern (missing fields, unexpected delimiters, truncated lines).
- The request field contains fewer than three whitespace-separated tokens, making method/path/protocol extraction impossible.
- Any numeric field (`status`, `bytes`, `hour`) throws a `NumberFormatException` during parsing.
- Any other unexpected exception is raised inside the `parse()` method.

Malformed records are *not* discarded during processing. Instead, each malformed log entry is preserved and stored in a dedicated PostgreSQL table named `malformed_record_summary`.

The table stores the original malformed record together with its corresponding batch identifier, allowing traceability, debugging, and validation of the ETL process.



	id [PK] integer	batch_id integer	record character varying (4096)
1	1	33	alyssa.p

Figure 3: Malformed Record inserted in Postgres

3.5 Pipeline Summary Output

At the end of a full pipeline run, `Main` prints a summary to stdout. The summary includes total records processed, valid and malformed counts, number of batches, average batch size, and total wall-clock execution time.

Table 3: Pipeline Summary — Full NASA Dataset Run

Metric	Value
Total Records Processed	3,461,613
Total Valid Records	3,461,612
Total Malformed Records	1
Total Batches	139
Average Batch Size	24,903.69
Total Execution Time (ms)	40,487

4. Relational Reporting Database Schema

4.1 Schema Design

The system uses PostgreSQL as the centralized relational reporting database. The schema is designed to store execution metadata, batch statistics, analytical query outputs, and malformed record information in a structured and consistent manner across all execution pipelines.

Separate result tables are maintained for each mandatory query in order to preserve clarity and simplify reporting. In addition, metadata tables are used to capture runtime statistics and execution details for every pipeline run.

4.1.1 Run Metadata Table

The `run_metadata` table stores information about every execution run performed by the system. Each run is assigned a unique `run_id`. The table records the selected pipeline, executed query, runtime, and execution timestamp.

Column	Data Type	Description
<code>run_id</code>	INTEGER	Unique identifier for each run
<code>pipeline_name</code>	VARCHAR(255)	Selected execution backend
<code>query_name</code>	INTEGER	Executed query identifier
<code>runtime</code>	NUMERIC(10,3)	Total execution runtime
<code>execution_timestamp</code>	TIMESTAMP WITH TIME ZONE	Timestamp of execution

Table 4: Run Metadata Schema

4.1.2 Batch Metadata Table

The `batch_metadata` table stores statistics related to batch processing. It records the number of processed records, malformed records, total batches, average batch size, and execution time for each run.

Column	Data Type	Description
run_id	INTEGER	Reference to execution run
pipeline_name	VARCHAR(255)	Pipeline used for execution
total_records	INTEGER	Total input records processed
total_valid	INTEGER	Total valid parsed records
total_malformed	INTEGER	Number of malformed records
total_batches	INTEGER	Number of batches processed
avg_batch_size	DOUBLE PRECISION	Average batch size
execution_time_ms	INTEGER	Execution time in milliseconds
timestamp	TIMESTAMP WITH TIME ZONE	Metadata insertion timestamp

Table 5: Batch Metadata Schema

4.1.3 Query Result Tables

Three separate tables are used to store the outputs of the mandatory analytical queries.

Query 1 Table The query_1 table stores the daily traffic summary grouped by log date and HTTP status code.

Column	Data Type	Description
log_date	DATE	Date of log entry
status_code	INT	HTTP response status code
request_count	BIGINT	Total number of requests
total_bytes	BIGINT	Total bytes transferred
batch_id	TEXT	Batch identifier

Table 6: Schema for Query 1 Results

Query 2 Table The query_2 table stores the top requested resources along with request statistics.

Column	Data Type	Description
resource_path	TEXT	Requested resource path
request_count	INT	Number of requests
total_bytes	BIGINT	Total bytes transferred
distinct_hosts	INT	Number of unique hosts
batch_id	TEXT	Batch identifier

Table 7: Schema for Query 2 Results

Query 3 Table The `query_3` table stores hourly error analysis statistics for HTTP error requests.

Column	Data Type	Description
<code>log_date</code>	TEXT	Date of log entry
<code>log_hour</code>	INT	Hour of request
<code>error_request_count</code>	INT	Number of error requests
<code>total_request_count</code>	INT	Total number of requests
<code>error_rate</code>	DOUBLE PRECISION	Ratio of error requests
<code>distinct_error_hosts</code>	INT	Unique hosts generating errors
<code>batch_id</code>	TEXT	Batch identifier

Table 8: Schema for Query 3 Results

4.1.4 Malformed Record Summary Table

Malformed records encountered during parsing are stored in the `malformed_record_summary` table instead of being discarded. This table preserves the original malformed log entry together with the corresponding batch identifier for debugging and validation purposes.

Column	Data Type	Description
<code>id</code>	INTEGER	Unique malformed record identifier
<code>batch_id</code>	INTEGER	Batch in which the record appeared
<code>record</code>	VARCHAR(4096)	Original malformed log entry

Table 9: Malformed Record Summary Schema

The schema design ensures consistency across all pipelines and provides a unified reporting layer for comparative analysis of execution behaviour, batching statistics, and query outputs.

5. Query Outputs and Demonstrations

5.1 Storage in PostgreSQL

Instead of returning results only to the console, the system directly inserts the aggregated output into PostgreSQL using a shared insertion service. This ensures persistence and enables further querying and reporting. Further, for identifying the pipeline run, we store the results of the queries in a database in postgresql named after the pipeline that was chosen for execution.

5.2 Query 1: Daily Traffic Summary

Query 1 computes the daily traffic summary by aggregating web server logs based on **log date** and **status code**. For each combination, the system calculates:

- Total number of requests
- Total number of bytes transferred

5.2.1 Implementation Details

The query is executed using a multi-stage approach:

- Each batch of logs is processed independently within the selected execution backend..
- Aggregation is performed independently on each batch.
- Partial aggregation outputs from all batches are combined to compute the final global aggregates..
- The final results are sorted by log date and status code.

5.2.2 Sample Output

	id [PK] integer	log_date date	status_code integer	request_count bigint	total_bytes bigint	batch_id text
1	1	1995-07-01	200	58033	1617409574	4+5+9
2	2	1995-07-01	302	2568	218465	4+5+9
3	3	1995-07-01	304	3797	0	4+5+9
4	4	1995-07-01	404	316	0	4+5+9
5	5	1995-07-02	200	54492	1531584293	1+5+6
6	6	1995-07-02	302	2370	195795	1+5+6
7	7	1995-07-02	304	3107	0	1+5+6
8	8	1995-07-02	403	5	0	1+5
9	9	1995-07-02	404	291	0	1+5+6
10	10	1995-07-03	200	81351	2085066566	3+6+16+39
11	11	1995-07-03	302	3122	261797	3+6+16+39
12	12	1995-07-03	304	4582	0	3+6+16+39
13	13	1995-07-03	403	2	0	6+16
14	14	1995-07-03	404	474	0	3+6+16+39
15	15	1995-07-03	500	53	0	39
16	16	1995-07-04	200	63069	1744266570	3+7+12
17	17	1995-07-04	302	2513	206813	3+7+12
18	18	1995-07-04	304	4507	0	3+7+12
19	19	1995-07-04	403	4	0	7
20	20	1995-07-04	404	359	0	3+7+12
21	21	1995-07-05	200	85080	2050959436	12+14+42+61
22	22	1995-07-05	302	3340	203410	12+14+42+61
23	23	1995-07-05	304	3340	203410	12+14+42+61
24	24	1995-07-05	403	3340	203410	12+14+42+61
25	25	1995-07-05	404	3340	203410	12+14+42+61
26	26	1995-07-05	500	3340	203410	12+14+42+61
27	27	1995-07-05	500	3340	203410	12+14+42+61
28	28	1995-07-05	500	3340	203410	12+14+42+61
29	29	1995-07-05	500	3340	203410	12+14+42+61
30	30	1995-07-05	500	3340	203410	12+14+42+61
31	31	1995-07-05	500	3340	203410	12+14+42+61
32	32	1995-07-05	500	3340	203410	12+14+42+61
33	33	1995-07-05	500	3340	203410	12+14+42+61
34	34	1995-07-05	500	3340	203410	12+14+42+61
35	35	1995-07-05	500	3340	203410	12+14+42+61
36	36	1995-07-05	500	3340	203410	12+14+42+61
37	37	1995-07-05	500	3340	203410	12+14+42+61
38	38	1995-07-05	500	3340	203410	12+14+42+61
39	39	1995-07-05	500	3340	203410	12+14+42+61
40	40	1995-07-05	500	3340	203410	12+14+42+61
41	41	1995-07-05	500	3340	203410	12+14+42+61
42	42	1995-07-05	500	3340	203410	12+14+42+61
43	43	1995-07-05	500	3340	203410	12+14+42+61
44	44	1995-07-05	500	3340	203410	12+14+42+61
45	45	1995-07-05	500	3340	203410	12+14+42+61
46	46	1995-07-05	500	3340	203410	12+14+42+61
47	47	1995-07-05	500	3340	203410	12+14+42+61
48	48	1995-07-05	500	3340	203410	12+14+42+61
49	49	1995-07-05	500	3340	203410	12+14+42+61
50	50	1995-07-05	500	3340	203410	12+14+42+61
51	51	1995-07-05	500	3340	203410	12+14+42+61
52	52	1995-07-05	500	3340	203410	12+14+42+61
53	53	1995-07-05	500	3340	203410	12+14+42+61
54	54	1995-07-05	500	3340	203410	12+14+42+61
55	55	1995-07-05	500	3340	203410	12+14+42+61
56	56	1995-07-05	500	3340	203410	12+14+42+61
57	57	1995-07-05	500	3340	203410	12+14+42+61
58	58	1995-07-05	500	3340	203410	12+14+42+61
59	59	1995-07-05	500	3340	203410	12+14+42+61
60	60	1995-07-05	500	3340	203410	12+14+42+61
61	61	1995-07-05	500	3340	203410	12+14+42+61
62	62	1995-07-05	500	3340	203410	12+14+42+61
63	63	1995-07-05	500	3340	203410	12+14+42+61
64	64	1995-07-05	500	3340	203410	12+14+42+61
65	65	1995-07-05	500	3340	203410	12+14+42+61
66	66	1995-07-05	500	3340	203410	12+14+42+61
67	67	1995-07-05	500	3340	203410	12+14+42+61
68	68	1995-07-05	500	3340	203410	12+14+42+61
69	69	1995-07-05	500	3340	203410	12+14+42+61
70	70	1995-07-05	500	3340	203410	12+14+42+61
71	71	1995-07-05	500	3340	203410	12+14+42+61
72	72	1995-07-05	500	3340	203410	12+14+42+61
73	73	1995-07-05	500	3340	203410	12+14+42+61
74	74	1995-07-05	500	3340	203410	12+14+42+61
75	75	1995-07-05	500	3340	203410	12+14+42+61
76	76	1995-07-05	500	3340	203410	12+14+42+61
77	77	1995-07-05	500	3340	203410	12+14+42+61
78	78	1995-07-05	500	3340	203410	12+14+42+61
79	79	1995-07-05	500	3340	203410	12+14+42+61
80	80	1995-07-05	500	3340	203410	12+14+42+61
81	81	1995-07-05	500	3340	203410	12+14+42+61
82	82	1995-07-05	500	3340	203410	12+14+42+61
83	83	1995-07-05	500	3340	203410	12+14+42+61
84	84	1995-07-05	500	3340	203410	12+14+42+61
85	85	1995-07-05	500	3340	203410	12+14+42+61
86	86	1995-07-05	500	3340	203410	12+14+42+61
87	87	1995-07-05	500	3340	203410	12+14+42+61
88	88	1995-07-05	500	3340	203410	12+14+42+61
89	89	1995-07-05	500	3340	203410	12+14+42+61
90	90	1995-07-05	500	3340	203410	12+14+42+61
91	91	1995-07-05	500	3340	203410	12+14+42+61
92	92	1995-07-05	500	3340	203410	12+14+42+61
93	93	1995-07-05	500	3340	203410	12+14+42+61
94	94	1995-07-05	500	3340	203410	12+14+42+61
95	95	1995-07-05	500	3340	203410	12+14+42+61
96	96	1995-07-05	500	3340	203410	12+14+42+61
97	97	1995-07-05	500	3340	203410	12+14+42+61
98	98	1995-07-05	500	3340	203410	12+14+42+61
99	99	1995-07-05	500	3340	203410	12+14+42+61
100	100	1995-07-05	500	3340	203410	12+14+42+61
101	101	1995-07-05	500	3340	203410	12+14+42+61
102	102	1995-07-05	500	3340	203410	12+14+42+61
103	103	1995-07-05	500	3340	203410	12+14+42+61
104	104	1995-07-05	500	3340	203410	12+14+42+61
105	105	1995-07-05	500	3340	203410	12+14+42+61
106	106	1995-07-05	500	3340	203410	12+14+42+61
107	107	1995-07-05	500	3340	203410	12+14+42+61
108	108	1995-07-05	500	3340	203410	12+14+42+61
109	109	1995-07-05	500	3340	203410	12+14+42+61
110	110	1995-07-05	500	3340	203410	12+14+42+61
111	111	1995-07-05	500	3340	203410	12+14+42+61
112	112	1995-07-05	500	3340	203410	12+14+42+61
113	113	1995-07-05	500	3340	203410	12+14+42+61
114	114	1995-07-05	500	3340	203410	12+14+42+61
115	115	1995-07-05	500	3340	203410	12+14+42+61
116	116	1995-07-05	500	3340	203410	12+14+42+61
117	117	1995-07-05	500	3340	203410	12+14+42+61
118	118	1995-07-05	500	3340	203410	12+14+42+61
119	119	1995-07-05	500	3340	203410	12+14+42+61
120	120	1995-07-05	500	3340	203410	12+14+42+61
121	121	1995-07-05	500	3340	203410	12+14+42+61
122	122	1995-07-05	500	3340	203410	12+14+42+61
123	123	1995-07-05	500	3340	203410	12+14+42+61
124	124	1995-07-05	500	3340	203410	12+14+42+61
125	125	1995-07-05	500	3340	203410	12+14+42+61
126	126	1995-07-05	500	3340	203410	12+14+42+61
127	127	1995-07-05	500	3340	203410	12+14+42+61
128	128	1995-07-05	500	3340	203410	12+14+42+61
129	129	1995-07-05	500	3340	203410	12+14+42+61
130	130	1995-07-05	500	3340	203410	12+14+42+61
131	131	1995-07-05	500	3340	203410	12+14+42+61
132	132	1995-07-05	500	3340	203410	12+14+42+61
133	133	1995-07-05	500	3340	203410	12+14+42+61
134	134	1995-07-05	500	3340	203410	12+14+42+61
135	135	1995-07-05	500	3340	203410	12+14+42+61
136	136	1995-07-05	500	3340	203410	12+14+42+61
137	137	1995-07-05	500	3340	203410	12+14+42+61
138	138	1995-07-05	500	3340	203410	12+14+42+61
139	139	1995-07-05	500	3340	203410	12+14+42+61
140	140	1995-07-05	500	3340	203410	12+14+42+61
141	141	1995-07-05	500	3340	203410	12+14+42+61
142	142	1995-07-05	500	3340	203410	12+14+42+61
143	143	1995-07-05	500	3340	203410	12+14+42+61
144	144	1995-07-05	500	3340	203410	12+14+42+61
145	145	1995-07-05	500	3340	203410	12+14+42+61
146	146	1995-07-05	500	3340	203410	12+14+42+61
147	147	1995-07-05	500	3340	203410	12+14+42+61
148	148	1995-07-05	500	3340	203410	12+14+42+61
149	149	1995-07-05	500	3340	203410	12+14+42+61
150	150	1995-07-05	500	3340	203410	12+14+42+61
151	151	1995-07-05	500	3340	203410	12+14+42+61
152	152	1995-07-05	500	3340	203410	12+14+42+61
153	153	1995-07-05	500	3340	203410	12+14+42+61
154	154	1995-07-05	500	3340	203410	12+14+42+61
155	155	1995-07-05	500	3340	203410	12+14+42+61
156	156	1995-07-05	500	3340	203410	12+14+42+61
157	157	1995-07-05	500	3340	203410	1

5.3 Query 2: Top Requested Resources

Query 2 computes the top 20 requested resource paths by aggregating web server logs based on **resource path**. For each selected resource, the system calculates:

- Total number of requests
- Total number of bytes transferred
- Number of distinct hosts requesting that resource

5.3.1 Implementation Details

The query is executed using a multi-stage approach:

- Each batch of logs is processed independently within the selected execution backend..
- Aggregation is performed independently on each batch.
- Each batch computes local request count, total bytes, and distinct hosts.
- Partial aggregation outputs from all batches are combined to compute the final global aggregates.
- The global top 20 resources are selected using request count ranking.
- The final output is displayed in ascending order of request count for readability.

5.3.2 Sample Output

	id [PK] integer	resource_path text	request_count integer	total_bytes bigint	distinct_hosts integer	batch_id text
1	1	/icons/blank.xbm	28827	13201424	17838	0+1+2+3+4+5+6+7+8+9+10+
2	2	/shuttle/missions/sts-69/sts-69-patch-small.gif	29101	212028657	18061	0+1+2+3+4+5+6+7+8+9+10+
3	3	/icons/menu.xbm	29166	13896463	17811	0+1+2+3+4+5+6+7+8+9+10+
4	4	/shuttle/countdown/liftoff.html	29839	111958733	14550	0+1+2+3+4+5+6+7+8+9+10+
5	5	/shuttle/missions/sts-69/mission-sts-69.html	31554	317116549	18161	0+1+2+3+4+5+6+7+8+9+10+
6	6	/htbin/cdt_main.pl	39839	143854352	12643	0+1+2+3+4+5+6+7+8+9+10+
7	7	/images/launchmedium.gif	40651	445305357	24823	0+1+2+3+4+5+6+7+8+9+10+
8	8	/shuttle/missions/missions.html	47263	379051096	27537	0+1+2+3+4+5+6+7+8+9+10+
9	9	/images/ksclogosmall.gif	61344	201833375	38812	0+1+2+3+4+5+6+7+8+9+10+
10	10	/	62863	397338535	31675	0+1+2+3+4+5+6+7+8+9+10+
11	11	/shuttle/countdown/	64696	254973856	36878	0+1+2+3+4+5+6+7+8+9+10+
12	12	/history/apollo/images/apollo-logo1.gif	68858	71737161	38380	1+7+9+14+16+17+19+21+22
13	13	/images/launch-logo.gif	75960	114606552	42524	0+1+2+3+4+5+6+7+8+9+10+
14	14	/ksc.html	83691	562133282	29146	0+1+2+3+4+5+6+7+8+9+10+
15	15	/images/ksclogo-medium.gif	121286	628025692	49419	0+1+2+3+4+5+6+7+8+9+10+
16	16	/images/WORLD-logosmall.gif	125676	73624119	53985	0+1+2+3+4+5+6+7+8+9+10+
17	17	/images/USA-logosmall.gif	126820	25932114	54429	0+1+2+3+4+5+6+7+8+9+10+
18	18	/images/MOSAIC-logosmall.gif	127656	40500273	54836	0+1+2+3+4+5+6+7+8+9+10+
19	19	/images/KSC-logosmall.gif	164813	169168020	85100	0+1+2+3+4+5+6+7+8+9+10+
20	20	/images/NASA-logosmall.gif	208437	131494656	81432	0+1+2+3+4+5+6+7+8+9+10+

Total rows: 20
Query complete 00:00:00.065

Figure 5: Query 2 output

5.3.3 Observations

- Static resources such as images, icons, and logo files dominate the ranking because browsers repeatedly request these assets.
- NASA logo images and homepage-related resources appear among the most requested entries, indicating frequent access to common landing pages.
- Large distinct host counts show that the most popular resources were accessed by many unique clients rather than a small number of repeated users.

5.4 Query 3: Hourly Error Analysis

Query 3 computes the hourly error analysis by aggregating web server logs based on **log date** and **log hour**. For each combination, the system calculates:

- Total number of error requests (HTTP status codes 400–599)
- Total number of requests

- Error rate percentage
- Number of distinct hosts generating error requests

5.4.1 Implementation Details

The query is executed using a multi-stage approach:

- Each batch of logs is processed independently within the selected execution backend..
- Aggregation is performed independently on each batch.
- Logs are grouped by `log_date` and `log_hour`.
- Error requests are identified using HTTP status codes in the range 400 to 599.
- Distinct hosts causing errors are collected.
- Partial aggregation outputs from all batches are combined to compute the final global aggregates.
- The final results are sorted by log date and hour.

5.4.2 Sample Output

	id [PK] integer	log_date text	log_hour integer	error_request_count integer	total_request_count integer	error_rate double precision	distinct_error_hosts integer	batch_id text
1	1	1995-07-01	0	24	3565	0.67	12	9
2	2	1995-07-01	1	10	3004	0.33	7	9
3	3	1995-07-01	2	12	2268	0.53	9	9
4	4	1995-07-01	3	16	1734	0.92	5	9
5	5	1995-07-01	4	10	1482	0.67	8	9
6	6	1995-07-01	5	9	1343	0.67	5	9
7	7	1995-07-01	6	4	1528	0.26	3	9
8	8	1995-07-01	7	2	1557	0.13	1	9
9	9	1995-07-01	8	6	1927	0.31	4	9
10	10	1995-07-01	9	3	2096	0.14	2	9
11	11	1995-07-01	10	13	3046	0.43	7	9
12	12	1995-07-01	11	23	3428	0.67	13	9
13	13	1995-07-01	12	10	2769	0.36	7	9
14	14	1995-07-01	13	13	3086	0.42	10	4+9
15	15	1995-07-01	14	19	3133	0.61	14	4
16	16	1995-07-01	15	17	3272	0.52	12	4
17	17	1995-07-01	16	14	3191	0.44	9	4
18	18	1995-07-01	17	15	3671	0.41	9	4
19	19	1995-07-01	18	15	3192	0.47	11	4
20	20	1995-07-01	19	16	2831	0.57	11	4
21	21	1995-07-01	20	10	2958	0.34	8	4
22	22	1995-07-01	21	24	2522	0.60	12	4
Total rows: 1369		Query complete 00:00:00.084						

Figure 6: Query 3 output

5.4.3 Observations

- Hours with higher error counts indicate peak failure periods in the system.
- The error rate highlights time intervals where failures form a significant portion of total traffic.
- Distinct error hosts help identify whether failures originate from many clients or a limited set of sources.
- Batch identifiers confirm that multiple batches contribute to the final aggregated results.
- The aggregation correctly combines results across all batches, validating the global merge logic.

6. Runtime Module

The Runtime Module is responsible for collecting and displaying execution statistics generated during ETL processing across all four pipelines.

A centralized PostgreSQL database named `global_db` is used to store runtime and batch-processing metadata for:

- MongoDB
- Hive
- Pig
- and Hadoop MapReduce

The framework maintains two important metadata tables:

- `run_metadata`
 - stores pipeline name,
 - executed query,
 - runtime,
 - and execution timestamps.
- `batch_metadata`
 - stores total records,
 - valid and malformed counts,
 - total batches,
 - average batch size,
 - and execution time statistics.

A dedicated `FinalReportService` fetches this information from PostgreSQL and prints formatted execution reports after pipeline completion.

Listing 17: Final Runtime Reporting

```
1 PostgresReaderService.readAllRowsPretty(  
2     "global_db",  
3     Arrays.asList("run_metadata")  
4 );  
5  
6 PostgresReaderService.readAllRowsPretty(  
7     "global_db",
```

```

8     Arrays.asList("batch_metadata")
9 );

```

Figure 7 shows the batch-processing statistics collected across all pipelines.

	run_id [PK] integer	pipeline_name character varying (255)	total_records integer	total_valid integer	total_malformed integer	total_batches integer	avg_batch_size double precision	execution_time_ms integer	timestamp timestamp with time zone
1	1	mongodb	3461613	3461612	1	35	98903.22857142857	136293	2026-05-18 18:43:31.598441+05:30
2	2	hive	3461613	3461612	1	35	98903.22857142857	243748	2026-05-18 18:50:19.533937+05:30
3	3	pig	3461613	3461612	1	35	98903.22857142857	145811	2026-05-18 18:55:44.421055+05:30
4	4	MapReduce	3461613	3461612	1	35	98903.23	38180	2026-05-18 19:01:00.511892+05:30

Figure 7: Batch Metadata Comparison

Figure 8 shows the runtime comparison between MongoDB, Hive, Pig, and Hadoop MapReduce pipelines.

	run_id [PK] integer	pipeline_name character varying (255)	query_name integer	runtime numeric (10,3)	execution_timestamp timestamp with time zone
1	1	mongodb	4	141896.000	2026-05-18 18:43:31.598441+05:30
2	2	hive	4	370737.000	2026-05-18 18:50:19.533937+05:30
3	3	pig	4	324834.000	2026-05-18 18:55:44.421055+05:30
4	4	MapReduce	4	264492.000	2026-05-18 19:01:00.511892+05:30

Figure 8: Runtime Comparison for all the pipelines

The Runtime Module therefore provides centralized monitoring and comparative performance analysis for the complete ETL framework.

7. Pipeline Performance Comparison

To evaluate the performance of the implemented ETL framework, execution statistics from all four pipelines were collected and compared using the centralized PostgreSQL reporting layer.

Table 10 presents the batch-processing statistics generated during execution.

Table 10: Batch Processing Comparison Across Pipelines

Pipeline	Record	Valid	Malformed	Batches	Avg Batch Size	Execution Time
MongoDB	3461613	3461612	1	35	98903.23	136293
Hive	3461613	3461612	1	35	98903.23	243748
Pig	3461613	3461612	1	35	98903.23	145811
MapReduce	3461613	3461612	1	35	98903.23	38180

Table 11 shows the overall runtime comparison collected from the `run_metadata` table.

Table 11: Overall Runtime Comparison

Pipeline	Query Executed	Runtime (ms)
MongoDB	4	141896
Hive	4	370737
Pig	4	324834
MapReduce	4	264492

All four pipelines successfully processed the same dataset and produced identical valid and malformed record counts, demonstrating consistency in parsing and validation across the framework.

From the batch-processing statistics, the Hadoop MapReduce pipeline achieved the fastest batch execution time with approximately **38180 ms**, followed by MongoDB and Pig, while Hive required the highest processing time among the four implementations.

In terms of overall runtime execution:

- MongoDB achieved the lowest overall runtime.
- Pig performed moderately with higher execution overhead caused by script-based processing.
- Hadoop MapReduce showed efficient distributed batch execution but incurred additional job orchestration overhead during complete query execution.
- Hive produced the highest runtime due to query compilation, HDFS staging, and distributed SQL execution overhead.

The comparison highlights the trade-offs between NoSQL storage systems, SQL-on-Hadoop frameworks, scripting-based analytics engines, and low-level distributed processing approaches for large-scale ETL workloads.

8. Conclusion

In this project, we designed and implemented a multi-pipeline NoSQL ETL framework for large-scale NASA HTTP log analytics using MongoDB, Hive, Pig, and Hadoop MapReduce.

The framework successfully demonstrated:

- scalable batch-based ETL processing,
- reusable shared parsing and validation logic,

- modular multi-module Maven architecture,
- distributed analytics execution,
- malformed record handling,
- and centralized PostgreSQL-based reporting.

All four pipelines produced consistent analytical outputs while highlighting the performance and architectural trade-offs between different NoSQL and distributed processing technologies.

The project also demonstrated how a common ETL workflow can be implemented across multiple big-data technologies while maintaining consistency, modularity, and extensibility.